

* pickling and un-pickling (or)

Object serialization or Object De-serialization: -

* Pickling (Object serialization): -

- Let us assume there exist an object which contains multiple values. To save or write an object data of main memory into the file of secondary memory by using `write()` and `writelines()`, they transfer the values in the form of value by value and it is one of the time consuming process (bcoz of multiple write operations).
- To overcome this time consuming process, we must use the concept of pickling.
- The Advantage of pickling concept is that with single write operation, we can save or write entire object data of main memory into the file of secondary memory.

* Definition of pickling: -

- The process of saving or transferring entire object content of main memory into the file of secondary memory by performing single write operation is called "pickling".
- pickling concept participates in write operation.

Steps for implementing pickling concept: -

- ⇒ Import pickle module, Here pickle is one of the pre-defined module.
- ⇒ Choose the file name and open it into write mode.
- ⇒ Create an object with collection of values (Iterable object)
- ⇒ Use the `dump()` of pickle module. `dump()` save the content of any object into the file with single write operation

Syntax: `pickle.dump(object, filepointer)`

Note: - pickling concept always takes the file in binary format

* Unpickling (Object De-serialization)

- let us assume there exists a record with multiple values in a file of secondary memory. To read or transfer the entire record content from file of secondary memory, if we use `read()`, `readlines()` then they read record values in the form of value by value and it is one of the time consuming process (bcoz of multiple read operations).
- To overcome this time consuming process, we must use the concept of unpickling.
- The Advantage of un-pickling is that with single read operation, we can read entire record-content from the file of secondary memory into the object of main memory.

* Definition of un-pickling:-

- the process of reading or transferring the entire record content from file of secondary memory into the object of main memory by performing single read operation is called "un-pickling".
- un-pickling concept participates in Read operations.

* Steps for implementing un-pickling concept:-

- ⇒ import pickle module
- ⇒ Choose the File name and open it into read mode.
- ⇒ Use the `load()` of pickle module. `load()` is used for transferring or loading the entire record content from file of secondary memory into object of main memory.

syntax : objname = pickle.load(file pointer)

→ Note:- unpickling concept always takes the file in Binary format.

Q. write a python program which will read the ~~data~~ student data from the keyboard and save as record present in a file

```
→ import pickle
def saverecord():
    with open("stud.pickle", "ab") as fp:
        while (True):
            # read student values from kbrd.
            print ( - - )
            sno = int (input("Enter student number"))
            sname =
            many =
            # create an empty list & add student values.
            l = list()
            l.append(sno)
            l.append(sname)
            l.append(many)
            # save list object content as a record to the
            # file
            pickle.dump(l, fp)
            print ( - - - )
            print ("Student Data saved as record:")
            print ( - - - )
            ch = input("Do u want to insert another)
                      record (Yes/No):")
            if (ch.lower() == "no"):
                break
```

```
# main program
saverecord()
```

Q. write a python program which will read the student records from the file (conpiscuing)

→ import pickle

```
try: with open("Address", "rb") as fp:
```

```
    print("-----")
```

```
    while (True):
```

```
        try:
```

```
            obj = pickle.load(fp)
```

```
            for val in obj:
```

```
                print("{} ".format(val), end="\t")
```

```
            print()
```

```
        except EOFError:
```

```
            print("-----")
```

```
            break
```

```
    # loop of code.
```

```
except FileNotFoundError:
```

```
    print("file does not exist")
```

Q. write a python program which will copy an image

→ try: with open("Address", "rb") as rp:

```
    with open("programmer.png", "wb") as wp:
```

```
        srcdata = rp.read() # reading the data from  
                             src file
```

```
        wp.write(srcdata) # writing.
```

```
        print("Image File Copied --- verify")
```

```
except FileNotFoundError:
```

```
    print("source file does not exists")
```

Q. write a python programmer which demonstrate random access file.

→ with open("addrl-data", "r") as fp:

print(" - - - - ")

print("Initial pos of fp =", fp.tell()) ~~≠ 0~~

- filedata = fp.read(5)

print("file data =", filedata)

print("Now pos of fp =", fp.tell()) ~~≠ 5~~

- filedata = fp.read(8)

→ same as.

filedata = fp.read()

→ same as.

fp.seek(6)

filedata = fp.read(7)

print("file data =", filedata)

print("Now pos of fp =", fp.tell())

filedata = fp.read()

print("file data =", filedata)

*working with csv files in python:-

- csv stands for comma separated values.
- A csv file is one of the simple file format used to store tabular data, such as a spreadsheet or database.
- A csv file stores tabular data (number or text) in a plain text.
- Each line of the csv file is a data record. Each record consists of one or more fields, separated by commas.
- Python provides an in-built module called csv to work with csv files.
- There are 2 classes provided by this module for writing the data to csv file.

They are:

- 1) By using csv.writer class object
- 2) By using csv.DictWriter class object

1) By using csv.writer class object:-

⇒ The csv.writer class object is used to insert data to the csv file.

⇒ To create an object of "csv.writer" class object, we use writer() and present in csv module.

⇒ "csv.writer" class object provides two functions for writing to csv file.

They are:

- 1) writerow()
- 11) writerows()

1) writerow() :- this method writes a single row at a time. Field row can be written using this method.

Syntax :- csvwriterobj.writerow (fieldsRow/Data Row)

1) writerows() :- This method is used to write multiple rows at a time.

→ This can be used to write rows list

syntax: writing csv files in python

csvWriterobj.writerows(data rows)

↓
Here data rows
can be list, tuple,
set, frozenset only

2) By using csv.DictWriter class object :-

→ The "csv.DictWriter" class object is used to insert dict data to the csv file.

→ To create an object of "csv.DictWriter" class object, we use DictWriter() and present in csv module.

→ "csv.DictWriter" class object provides two functions for writing to csv.

i) writeheader()

ii) writerows()

i) writeheader() :- writeheader() method simply writes the first row of your csv file using the pre-defined specified fieldnames.

syntax: DictWriterobj.writeheader()

ii) writerows() :- writerows() method simply writes all the value of (key, value) from dict object in the form of separate rows.

[Note: It writes only the values (not keys)]

syntax: DictWriterobj.writerows(dictobj)

* Reading the data from CSV File:-

→ There are various ways to read a CSV file that uses either the CSV module or the pandas library

→ The CSV module provides classes for reading information from CSV file.

1) CSV.reader

2) CSV.DictReader.

1) CSV.reader() :- This function is used for creating an object of "CSV.reader" class and it helps us to read the data records from CSV file.

syntax:

csvreaderobj = csv.reader(filepointer),

2) CSV.DictReader() :- This function is used for creating an object of "CSV.DictReader" class and it helps us to read the data from CSV file where it contains dict data (key, value)

syntax:

csvdictreaderobj = csv.DictReader(file pointer)

Program for reading the data from CSV file by using file pointer

Non CSV

by:

with open ("stud.csv", "r") as fp:

print (" — — — — — ")

filedata = fp.read()

print (filedata)

print (" — — — — — ")

except FileNotFoundError:

print ("file does not exist")

program for creating csv file by python program with csv module.

```
import csv # step 1
```

```
hnames = ["empno", "ename", "sal", "dsg"] # step 2
```

```
records = [ [10, "RS", 3.4, "Author"],  
             [20, "TR", 4.4, "Scientist"],  
             [30, "DR", 1.4, "SE"],  
             [40, "SJ", 2.4, "TL"],  
             [50, "GS", 1.4, "HR"] ] # step 3
```

```
with open("emp.csv", "a") as fp: # step 4
```

```
    csvwr = csv.writer(fp) # step 5 - Hse csvwr  
                                is an object of <class, csv.  
                                writer>
```

```
# Hse write Header names to the csv file...
```

```
    csvwr.writerow() present in object of  
    <class, csv>
```

```
    csvwr.writerow(hnames) # step 6
```

```
# write the records which are in the form nested list.
```

```
    csvwr.writerows(records) # step 7
```

```
    print("csv file created and verify")
```

program for Adding record to csv file by python program with csv module.

```
→ import csv  
    record csv # step 1
```

```
record = [60, "KVR", 0.0, "Trainer"]
```

```
with open("emp.csv", "a") as fp:
```

```
    csvwr = csv.writer(fp)
```

```
    csvwr.writerow(record)
```

```
    print("record saved in a file.")
```

Program for adding record to CSV File by Python program with csv module.

```
import csv # step 1
```

```
with open ("emp.csv", "a") as fp:
```

```
    while (True):
```

```
        print(" — — — — — ")
```

```
        # accept the employee values from KBD
```

```
        eno = int(input("Enter employee number: "))
```

```
        ename = input("Enter employee name: ")
```

```
        sal = float(input("Enter employee salary: "))
```

```
        dsgr = input("Enter employee Designation: ")
```

```
        # create an empty list
```

```
        lst = list()
```

```
        # append employee Data to list object
```

```
        lst.append(enr)
```

```
        lst.append(ename)
```

```
        lst.append(sal)
```

```
        lst.append(dsgr)
```

```
        # Create an object csv.writer object
```

```
        csvwr = csv.writer(fp)
```

```
        # write record data to csv file
```

```
        csvwr.writerow(lst)
```

```
        print("Record saved in a file")
```

```
        print(" — — — — — ")
```

```
        ch = input("Do u want to insert another record (y/n): ")
```

```
        if (ch.lower() == "n"):
```

```
            break
```

program for reading the data from csv file by using csv module

```
import csv
```

```
try: with open("std.csv", "r") as fp:
```

```
    print(" - - - - - ")
```

ex①

```
    csvr = csv.reader(fp) # here csvr is an object  
                           of <class, csv.reader>
```

```
    for record in csvr:
```

```
        for val in record:
```

```
            print("{} ".format(val), end="|")
```

```
        print()
```

```
    print(" - - - - - ")
```

```
except FileNotFoundError:
```

```
    print("file does not exist")
```

ex②). ~~import~~ isme ("emp.csv", "r").

program for reading the data from CSV File by using csv module
in the form Dict

```
import csv
```

```
with open("emp.csv", "r") as fp
```

```
    csvdr = csv.DictReader(fp) # Here csvdr is an object  
                               of type <class, csv.DictReader>
```

ex①

```
    for drdata in csvdr:
```

```
        for k, v in drdata.items():
```

```
            print("{} ---> {}".format(k, v))
```

```
    print(" - - - - - ")
```

ex②) isme "std.csv"

Program for creating CSV file by Python with csv module where the data is available in the form of dict

import csv # step 1

hname = ["HTNO", "NAME", "BRANCH", "CGPA"] # step 2

records = [{ "HTNO": 100, "NAME": "VIVEK", "BRANCH": "CSE", "CGPA": 7.8 }
200, nitesh, CCE, 7.8
300, Anish,
400, Anmol
500, Sujita, # step 3

with open ("univstud.csv", "a") as fp: # step 4

csvwdr = csv.DictWriter (fp, fieldnames = hname). # step 5

csvwdr = writerheader () # step 6

csvwdr. writerows (records) # step 7

print ("Dict Data saved as records in csv file")

Day (62)

* Working with OS Based operations:-

→ In Python, 'os' is pre-defined module.

→ The purpose of os module is that "To perform some os related operations".

→ The OS based operations are:-

- 1) Creating folder/Directory (mkdir())
- 2) Creating Folder/Hierarchy (mkdirs())
- 3) Removing folder/Directory (rmdir())
- 4) Removing Folder Hierarchy (removedirs())
- 5) Removing file name from folder (remove())
- 6) Renaming a folder/file Name (rename())
- 7) List the file names in folder (listdir())

1) Creating Folder / Directory

→ For creating a folder / Directory, we use `mkdir()`.

→ syntax: `os.mkdir("Folder Name")`

⇒ if the Folder name already exist then we get FileExistsError.

⇒ `mkdir()` can create only one Folder at a time and if we try to create folders hierarchy then we get FileNotFoundError.

⇒ in `mkdir()`, if we specify any folder name with ~~escape~~ escape sequence (`\n \u \digit \t`.... etc.) then we get osError.

program for creating a single folder.

```
import os
```

```
try:
```

```
    os.mkdir("C:\\INDRA\\ITS")
```

```
    print("Folder created successfully")
```

```
except FileExistsError:
```

```
    print("Folder Already created")
```

```
except FileNotFoundError:-
```

```
    print("Folder does not exist")
```

2) Creating Folders hierarchy:-

→ For creating Folders hierarchy, we use `makedirs()`

→ syntax: `os.makedirs("Folders hierarchy")`

⇒ Here folders hierarchy represent root folder
| sub folder | sub-sub folder & so on . . .

⇒ if the folder name already exist then we get FileExistsError.

⇒ if we specify any folder name with escape sequence (`\n \u \digit \t`.... etc.) then we get osError.

program for creating Folder Hierarchy:-

```
x
import os
try:
    os.makedirs("D:\KVR\H5D\PYTHON")
    print("Folder hierarchy created -- verify")
except FileExistsError:
    print("Folder hierarchy already exist")
```

3) Removing Folder/Directory

→ For Removing Folder/Directory, we use `rmdir()`

→ syntax : `os.rmdir("folder name")`

→ `rmdir()` can remove folder name provided name is empty.

→ if we specify any folder name with escape sequence (`\n \u \digits, \t....etc`) then we get `osError`.

program for deleting a single folder:-

```
import os
try:
    os.rmdir("APPLE")
    print("Folder Deleted -- verify")
except FileNotFoundError:
    print("Folder does not exist")
except osError:
    print("Folder is not empty")
```

4) . Removing Folder/Hierarchy:-

→ For Removing Folders Hierarchy, we use `removedirs()`

→ syntax:- `os.removedirs("Folders Hierarchy")`

→ These folders hierarchy represent Root Folder/sub Folder/
sub-sub Folder so on...

→ if the folder name not exist then we get `FileNotFoundError`

→ if we specify any folder name with escape sequence (\n \v \d etc) then we get os Error.

program for removing the Folders hierarchy

```
import os
```

```
try:
```

```
    os.removedirs("C:\\KVR\\PYTHON")
```

```
    print("Folder hierarchy removed --- verify")
```

```
except FileNotFoundError:
```

```
    print("Folder Does not Exist")
```

```
except osError:
```

```
    print("Folder is not empty")
```

5.) Removing File Name from Folder

→ to remove the file name from Folder, we use remove()

→ syntax :- os.remove("Absolute path of File Name")

→ if the file name does not exist then we get FileNotFoundError

program for removing a file

```
import os
```

```
try:
```

```
    os.remove("D:\\sub\\kvr.py")
```

```
    print("File name removed --- verify")
```

6.) Renaming a folder/File Name:

x

→ to rename a folder, we rename()

→ syntax :- os.rename("old Folder Name", "New Folder Name")

→ syntax :- os.rename("old Folder Name", "New Folder Name")

→ if the old folder name does not exist then we get FileNotFoundError.

program for renaming a File

```
import os
try:
    os.rename("D:\\SUB\\kv.py", "D:\\SUB\\hxd.py")
    print("File name renamed --- verify")
except FileNotFoundError:
    print("source File Name does not exist")
```

program for renaming a Folder .

```
import os
try:
    os.rename("D:\\KVR", "D:\\rossum")
    print("Folder Renamed --- verify")
except FileNotFoundError:
    print("source folder does not exist")
```

7.) List the file names in folder:-

x → to list the file names in folder, we use `listdir()`
→ syntax: `os.listdir("Absolute path of Folder Name")`
→ if the Folder Name does not exist then we get File Not found Error.

program for listing the files in a Folder

```
import os
Filelist = os.listdir("E:\\KVR-PYTHON-7AM\\CSV")
print("-----")
for file in Filelist:
    print("\t{}".format(file))
print("-----")
print("Number of Files =", len(Filelist))
print("-----")
```

ex 10

ex(2). import os
by:

folderName = input("Enter the folder name to list the Files:")

ex(2) Filelist (बाकी सब सम)

except FileNotFoundError:

print("Folder does not exist")

ex(3)

ex(3) print ke baad

noP = 0

for file in Filelist:

if (file.endswith(".PY")):

print("\t{0}".format(file))

noP = noP + 1

print("— — — — —")

print("Total number of Files =", len(Filelist))

print("Number of Python Files =", noP)

print("— — — — —")

except FileNotFoundError:

print("Folder does not exist")

ex(4)

noP = 0

for file in Filelist:

if (file[-4] == ".csv"):

ex(5)

if (file._contains_(".csv"))